

DETECCIÓN DE OUTLIERS

Nombre: Steven Chuiza

Fecha: 05/05/2026

```
In [2]: import pandas as pd
import numpy as np
from scipy import stats
from sklearn.covariance import EllipticEnvelope
import matplotlib.pyplot as plt
```

```
In [4]: # Carga de datos.
df = pd.read_csv("dataset/outliers.csv")
print(df)
```

	a	b
0	0.149014	-0.041479
1	0.194307	0.456909
2	-0.070246	-0.070241
3	0.473764	0.230230
4	-0.140842	0.162768
..	...	...
195	-5.114441	0.646251
196	5.631630	0.277174
197	1.552784	2.348984
198	-0.545507	1.530697
199	1.011772	4.813896

[200 rows x 2 columns]

1) EllipticEnvelope()

```
In [5]: # Entrenamos un objeto de tipo EllipticEnvelope
algorithm = EllipticEnvelope(support_fraction=None, contamination=0.25, random_s
outlier_method = algorithm.fit(df)

# Aplicamos el método de detección de outliers entrenado sobre nuestro dataset de
df_outliers = outlier_method.predict(df)
print(df_outliers)

# # Determinar La posición de los outliers
pos_outliers = np.where(df_outliers==-1)[0]
print('\nOutliers en la posición: \n', pos_outliers)

# # # Determinar el número de outliers
print('\nNúmero de outliers: \n', len(pos_outliers))
```

```
[ 1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1
  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1
  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1
  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1
  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1
  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1
  1  1  1  1  1  1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1
 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1
 -1 -1 -1 -1 -1 -1 -1 -1]
```

Outliers en la posición:

```
[150 151 152 153 154 155 156 157 158 159 160 161 162 163 164 165 166 167
168 169 170 171 172 173 174 175 176 177 178 179 180 181 182 183 184 185
186 187 188 189 190 191 192 193 194 195 196 197 198 199]
```

Número de outliers:

50

```
In [6]: # Definimos una función que, dado un determinado "df" y un "algorithm", devuelva
def find_outliers(df, algorithm):

    #Caso especial: LocalOutlierFactor
    if algorithm.__class__.__name__ == "LocalOutlierFactor":
        df_outliers = algorithm.fit_predict(df)
    else:
        #Para otros algoritmos
        outlier_method = algorithm.fit(df)
        #Aplicamos el método de detección de outliers entrenado sobre nuestro d
        df_outliers = outlier_method.predict(df)

    print(df_outliers)
    #determinamos la posición de los outliers
    pos_outliers=np.where(df_outliers== -1)[0]
    print('\nOutliers en la posición :\n', pos_outliers)
    #Determinamos el número de outliers
    print('\nNúmero de outliers: \n', len(pos_outliers))

    return df_outliers, pos_outliers
```

2) Otros métodos similares

```
In [7]: from sklearn.ensemble import IsolationForest
from sklearn.svm import OneClassSVM
from sklearn.neighbors import LocalOutlierFactor

IF = IsolationForest(max_samples='auto', random_state=42)
OC_SVM = OneClassSVM(kernel="poly", degree=1, gamma="auto")
LOF = LocalOutlierFactor(n_neighbors=10, algorithm="auto", metric="euclidean")

df_outliers, pos_outliers = find_outliers(df, OC_SVM)
print(len(pos_outliers))
```

```
[ 1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1
  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1
  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1
  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1
  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1
  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1
  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1
  1  1  1  1  1  1  1  1 -1 -1 -1 -1 -1 -1  1 -1 -1  1  1 -1 -1  1  1 -1 -1
  1  1 -1  1  1  1  1  1  1  1  1  1 -1 -1  1 -1 -1  1  1  1  1 -1 -1 -1
  1  1 -1  1  1 -1 -1  1 -1]
```

Outliers en la posición :

```
[151 152 153 154 155 156 158 159 162 163 166 167 170 179 180 182 183 188
 189 190 191 194 196 197 199]
```

Número de outliers:

25

25

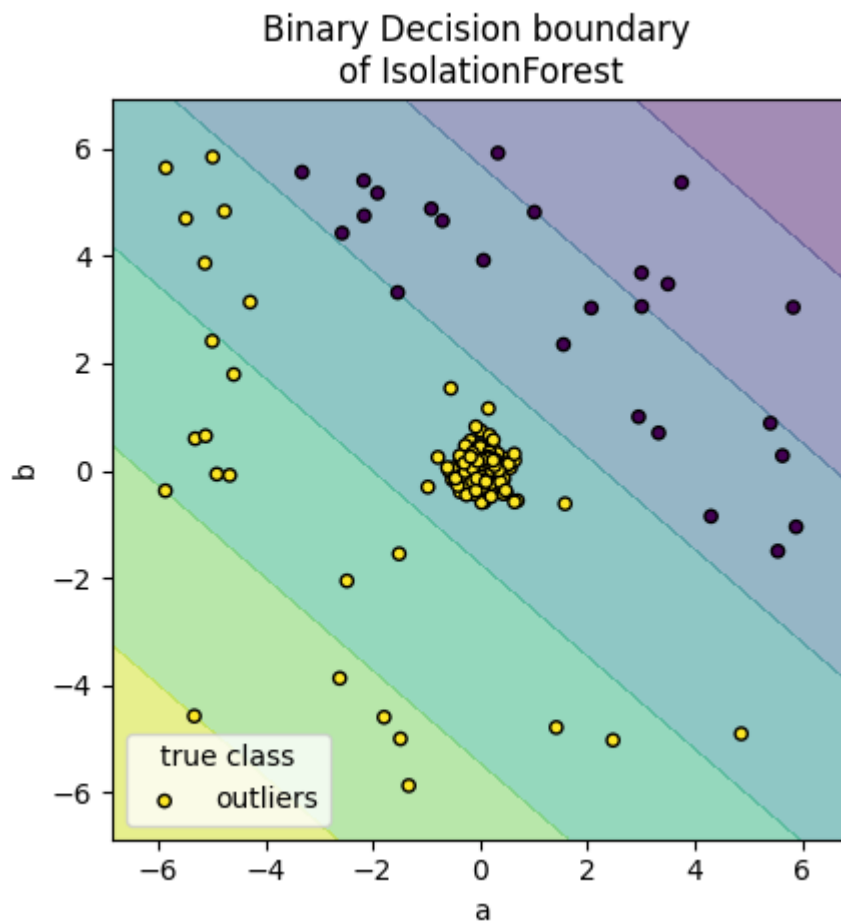
In [8]: `df.head(10)`

```
Out[8]:
```

	a	b
0	0.149014	-0.041479
1	0.194307	0.456909
2	-0.070246	-0.070241
3	0.473764	0.230230
4	-0.140842	0.162768
5	-0.139025	-0.139719
6	0.072589	-0.573984
7	-0.517475	-0.168686
8	-0.303849	0.094274
9	-0.272407	-0.423691

```
In [9]: import matplotlib.pyplot as plt
from sklearn.inspection import DecisionBoundaryDisplay

disp = DecisionBoundaryDisplay.from_estimator(
    OC_SVM,
    df,
    response_method = "decision_function",
    alpha=0.5,
)
disp.ax_.scatter(df.a,df.b,c=df_outliers,s=20,edgecolor="k")
disp.ax_.set_title("Binary Decision boundary \nof IsolationForest")
plt.axis("square")
plt.legend(labels=["outliers","inliers"], title="true class")
plt.show()
```



```
In [10]: # Eliminamos los outliers
new_df = df[df_outliers==1]
print(new_df)
```

	a	b
0	0.149014	-0.041479
1	0.194307	0.456909
2	-0.070246	-0.070241
3	0.473764	0.230230
4	-0.140842	0.162768
..	...	...
187	-4.589685	1.790524
192	-5.854146	5.638546
193	-5.482081	4.693717
195	-5.114441	0.646251
198	-0.545507	1.530697

[175 rows x 2 columns]

3) Box plot

```
In [11]: # Seleccionamos el atributo que vamos a medir
a = df['a']

# Seleccionamos los umbrales a partir de los cuales vamos a considerar outliers
Q1 = stats.scoreatpercentile(a, 25)
Q3 = stats.scoreatpercentile(a, 75)
RIC = Q3 - Q1
li = Q1 - 1.5*RIC #xmin
ls = Q3 + 1.5*RIC #xmax
```

```

# Observamos los límites inferior y superior
print('límite inferior: ', li)
print('límite superior: ', ls)

# Buscamos la posición de los outliers
pos_i = np.where(a<li)[0]
pos_s = np.where(a>ls)[0]
pos_outliers = np.concatenate((pos_i, pos_s))
print('Posición de outliers: ', pos_outliers)
print('Número de outliers: ', len(pos_outliers))

# Dibujamos el diagrama de caja y bigotes
prop = plt.boxplot(a)
plt.boxplot(a)
plt.show()

```

límite inferior: -1.0853060827395105

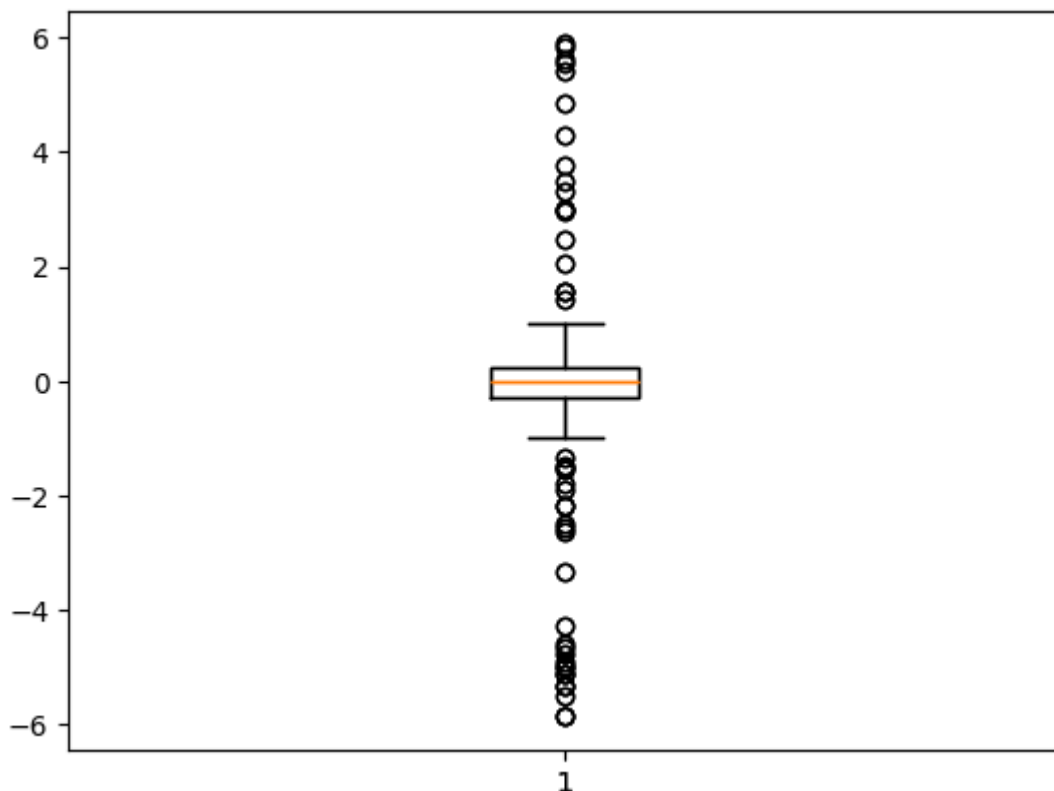
límite superior: 1.0119554520872196

Posición de outliers: [150 153 154 157 159 160 162 165 168 169 171 172 174 175 177 178 181 184

185 186 187 190 191 192 193 195 151 152 155 156 161 163 164 166 167 173

176 179 180 182 188 189 196 197]

Número de outliers: 44



```

In [12]: # Definir una función que, dada una columna de un dataframe, devuelva la posición
def find_limits_BP(variable):
    # 1. Calculamos los cuartiles 1 y 3
    q1 = variable.quantile(0.25)
    q3 = variable.quantile(0.75)

    # 2. Calculamos el Rango Intercuartílico (IQR)
    iqr = q3 - q1

    # 3. Definimos los límites (vallas) superior e inferior
    lim_inf = q1 - 1.5 * iqr

```

```

lim_sup = q3 + 1.5 * iqr

# 4. Filtramos las posiciones donde el valor está fuera de los límites
# Usamos .index para obtener la posición y .tolist() para devolver una lista
pos_outliers = variable[(variable < lim_inf) | (variable > lim_sup)].index.tolist()

return pos_outliers

```

```

In [13]: # Creamos un bucle for que estime los valores outliers de cada atributo
headers = df.columns # nombre de los atributos del CSV
pos_outliers = []
for i in range(len(headers)):
    variable = df[headers[i]] # Atributo 'x'
    pos_out = np.expand_dims(find_limits_BP(variable), axis=1) # Llamamos a la f
    pos_outliers.append(pos_out) # Lo añadimos en una lista

# Concatenamos todas las posiciones de outliers
po = np.vstack(pos_outliers)

# Vemos las posiciones de todos los outliers
pos_out = np.unique(po)
print('Posiciones de outliers: ', pos_out)

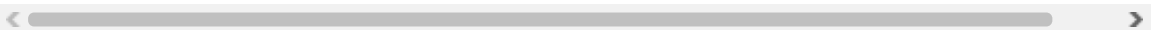
# Observamos el número de outliers
print('Numero de outliers: ', len(pos_out))

```

Posiciones de outliers: [104 150 151 152 153 154 155 156 157 158 159 160 161 162
163 164 165 166
167 168 169 170 171 172 173 174 175 176 177 178 179 180 181 182 183 184
185 186 187 188 189 190 191 192 193 194 195 196 197 198 199]
Numero de outliers: 51

Repositorio de

GitHub: https://github.com/tiven29/Fundamentos_Machine_Learning/tree/main/.ipynb_check



In []: